Digital Wallet Simulation (Encapsulation-Based)

By
Manan Jain
150096725166

01

CASE STUDY

Problem Statement

DigiWallet
Simulation Project

01 No Data Security

Wallet balance was directly accessible and modifiable from the browser console — no protection against external tampering.

02 No Instant Feedback

Traditional apps require page reloads to reflect transaction results, creating a slow and unresponsive user experience.

03 No Transaction History

Users had no way to track credits or debits — no ledger was maintained or displayed after transactions occurred.

Our Solution

DigiWallet — Encapsulation-Based Digital Wallet

🔒 Encapsulated Wallet Class

Balance and transaction list are private variables. Only topUp() and makePayment() methods can access or modify them — not the browser console.

✅ Smart Input Validation

Every amount is checked with isNaN() and amount > 0. Payments that exceed the available balance are blocked, preventing overdrafts.

⚡ Real-Time DOM Updates

A dedicated updateUI() function runs after every transaction, instantly syncing the displayed balance and history — zero page reloads.

📋 Transaction Ledger

Each transaction is saved as a Transaction object with an auto-generated timestamp and rendered in reverse-chronological order.

How It Works

Step-by-step workflow of the DigiWallet system

1 Login

User enters their name. JavaScript creates a new Wallet() instance, initializing private balance and history.

→

2 Top-Up

Amount is validated. If valid, _balance increases and a Top Up object is pushed to _transactions.

→

3 Payment

Balance is checked first. If sufficient, _balance decreases. If not, UI shakes and an error modal appears.

→

4 History

renderHistory() spreads and reverses the array, then injects dynamic <li> elements into the DOM.

Conclusion

Key findings and outcomes from the DigiWallet project

Key Takeaways

- Encapsulation keeps balance secure — inaccessible from the browser console
- Animations (balance-pop, headShake) make the UI feel premium and responsive
- parseFloat validation prevents NaN errors in all financial calculations
- Class-based design makes the codebase easy to scale and maintain
- DOM sync via updateUI() bridges backend logic and frontend display seamlessly

100%

Balance Security

via Encapsulation

0

Page Reloads

Real-time DOM sync

ES6

JavaScript Classes

OOP-first design

ITM SKILLS UNIVERSITY

Thank You